

MỤC LỤC

1. Kiểu dữ liệu	4
1.1. Đối tượng Vectors	4
1.2. R-Object	5
1.2.1. Vectors	5
1.2.2. Lists	5
1.2.3. Ma trận	5
1.2.4. Arrays	6
1.2.5. Factors (Các nhân tố)	6
1.2.6. Data Frames	7
2. Biến	7
2.1. Gán biến	8
2.2. Tìm biến	8
2.3. Xóa biến	9
3. Toán tử	9
3.1. Toán tử số học	9
3.2. Toán tử quan hệ	11
3.3. Toán tử logic	12
3.4. Assignment Operators	13
3.5. Miscellaneous Operators	14
4. Decision making	15
4.1. If-else statement	15
4.1.1. Cú pháp	15
4.1.2. Ví dụ	15
4.2. Switch Statement	16
4.2.1. Cú pháp	16
4.2.2. Ví dụ	16
5. Vòng lặp	16
5.1. For Loop	16
5.1.1. Syntax	16
5.1.2. Example	16
5.2. While Loop	17
5.2.1. Cú pháp	17
5.2.2. Ví dụ	17

5.3.	Loop Control Statements	17
5.3.1.	Break.....	17
5.3.2.	Next.....	18
6.	Function.....	18
6.1.	User-defined and Calling a Function.....	18
6.2.	Một vài hàm cơ bản.....	19
6.2.1.	Toán học	19
6.2.2.	Tính toán ma trận.....	19
6.2.3.	Thống kê mô tả.....	20
6.2.4.	Khác	20
7.	Dữ liệu	20
7.1.	Nhập dữ liệu	20
7.2.	Xử lý dữ liệu	21
7.3.	Thao tác với dữ liệu.....	21
7.3.1.	Xử lý dữ liệu bị thiếu.....	21
7.3.2.	Trích dữ liệu	22
7.4.	Họ các hàm apply.....	22
7.4.1.	apply.....	22
7.4.2.	lapply() và sapply()	23
8.	Trình bày và mô tả dữ liệu.....	23
8.1.	Các khái niệm cơ bản.....	23
8.1.1.	Các số đo xu thế trung tâm.....	23
8.1.2.	Các số đo mức độ phân tán.....	23
8.2.	Phát hiện các giá trị điểm bất thường.....	23
9.	Trình bày dữ liệu	24
9.1.	Một số khả năng.....	24
9.1.1.	Các phương pháp biểu diễn dữ liệu:	24
9.1.2.	Các số đo xu thế trung tâm:.....	24
9.1.3.	Các số đo mức độ phân tán:.....	25
9.2.	Biểu đồ cột.....	25
9.2.1.	Cách vẽ	25
9.2.2.	Biểu đồ cột khi thêm thông số labels, title và colors.....	26
9.3.	Biểu đồ cột kép và biểu đồ cột chồng.....	27
9.4.	Histograms.....	28

9.4.1.	Cách vẽ	28
9.4.2.	Biểu đồ khi cho giới hạn giá trị X và Y.....	30
9.5.	Biểu đồ đường	31
9.5.1.	Cách vẽ	31
9.5.2.	Hàm biểu đồ đường với các thông số title, color và labels.....	32
9.5.3.	Biểu đồ chứa nhiều đường.....	33
9.6.	Biểu đồ tròn	34
9.6.1.	Cách vẽ	34
9.6.2.	Pie Chart Title and Colors.....	35
9.6.3.	Slice Percentages and Chart Legend	36
9.6.4.	3D Pie Chart	37
9.7.	Biểu đồ Boxplot.....	38
9.7.1.	Cách vẽ	38
9.7.2.	Tạo biểu đồ boxplot với notch (rãnh).....	40
9.8.	Cách vẽ nhiều biểu đồ trong R.....	41
9.8.1.	Sử dụng par()	41
9.8.2.	Sử dụng layout().....	43

1. Kiểu dữ liệu

1.1. Đối tượng Vectors

Data Type	Example	Verify
Logical	TRUE, FALSE	<pre>v <- TRUE print(class(v))</pre> Kết quả <pre>[1] "logical"</pre>
Numeric	12.3, 5, 999	<pre>v <- 23.5 print(class(v))</pre> Kết quả <pre>[1] "numeric"</pre>
Integer	2L, 34L, 0L	<pre>v <- 2L print(class(v))</pre> Kết quả <pre>[1] "integer"</pre>
Complex	3 + 2i	<pre>v <- 2+5i print(class(v))</pre> Kết quả <pre>[1] "complex"</pre>
Character	'a' , '"good", "TRUE", '23.4'	<pre>v <- "TRUE" print(class(v))</pre> Kết quả <pre>[1] "character"</pre>
Raw	"Hello" is stored as 48 65 6c 6c 6f	<pre>v <- charToRaw("Hello") print(class(v))</pre> Kết quả <pre>[1] "raw"</pre>

1.2. R-Object

1.2.1. Vectors

```
# Tạo 1 vector.  
apple <- c('red','green',"yellow")  
print(apple)  
  
# Get the class of the vector.  
print(class(apple))
```

Khi thực hiện đoạn code code, Kết quả

```
[1] "red"      "green"    "yellow"  
[1] "character"
```

1.2.2. Lists

```
# Create a list.  
list1 <- list(c(2,5,3),21.3,sin)  
  
# Print the list.  
print(list1)
```

Khi thực hiện đoạn code code, Kết quả

```
[[1]]  
[1] 2 5 3  
  
[[2]]  
[1] 21.3  
  
[[3]]  
function (x) .Primitive("sin")
```

1.2.3. Ma trận

```
# Create a matrix.  
M = matrix( c('a','a','b','c','b','a'), nrow = 2, ncol = 3, byrow =  
TRUE)  
print(M)
```

Khi thực hiện đoạn code code, Kết quả

```
      [,1] [,2] [,3]  
[1,] "a"  "a"  "b"  
[2,] "c"  "b"  "a"
```

Lưu ý: nrow và ncol là số lượng phần tử theo hàng và cột. byrow sẽ xếp các phần tử vào từng hàng nếu TRUE và ngược lại mặc định là FALSE.

1.2.4. Arrays

```
# Create an array.  
a <- array(c('green','yellow'),dim = c(3,3,2))  
print(a)
```

Khi thực hiện đoạn code code, Kết quả

```
, , 1  
  
      [,1]      [,2]      [,3]  
[1,] "green"  "yellow" "green"  
[2,] "yellow" "green"  "yellow"  
[3,] "green"  "yellow" "green"  
  
, , 2  
  
      [,1]      [,2]      [,3]  
[1,] "yellow" "green"  "yellow"  
[2,] "green"  "yellow" "green"  
[3,] "yellow" "green"  "yellow"
```

1.2.5. Factors (Các nhân tố)

Các yếu tố là các đối tượng-R được tạo bằng cách sử dụng một vector. Nó lưu trữ vector cùng với các giá trị riêng biệt của phần tử bất kể đó là số hay kí tự hay Boolean,... trong vecto đầu vào. Chúng rất hữu ích trong mô hình thống kê.

Các factor(yếu tố) được tạo bằng cách sử dụng hàm **factor()**. Hàm **nlevels** đưa ra số lượng cấp độ.

```
# Create a vector.  
apple_colors <-  
c('green','green','yellow','red','red','red','green')  
  
# Create a factor object.  
factor_apple <- factor(apple_colors)  
  
# Print the factor.  
print(factor_apple)  
print(nlevels(factor_apple))
```

Khi thực hiện đoạn code code, Kết quả

```
[1] green green yellow red red red green  
Levels: green red yellow  
[1] 3
```

1.2.6. Data Frames

Data frames (Khung dữ liệu) là các đối tượng dạng bảng. Không giống như ma trận trong Data frames, mỗi cột có thể chứa các chế độ dữ liệu khác nhau. Cột đầu tiên có thể là số trong khi cột thứ hai có thể là kí tự và cột thứ 3 là logic. Nó là một danh sách các vecto có độ dài bằng nhau.

Data Frames được tạo bằng hàm `data.frame()`.

```
# Khởi tạo data frame.
BMI <- data.frame(
  gender = c("Male", "Male", "Female"),
  height = c(152, 171.5, 165),
  weight = c(81, 93, 78),
  Age = c(42, 38, 26)
)
print(BMI)
```

Khi thực hiện đoạn code code, Kết quả

```
  gender height weight Age
1  Male   152.0     81  42
2  Male   171.5     93  38
3 Female   165.0     78  26
```

2. Biến

Tên biến	Validity(Hợp lệ)	Lí do
var_name2.	valid(có hợp lệ)	Có chữ cái, số, dấu chấm và gạch dưới.
var_name%	Invalid (không hợp lệ)	Có kí tự '%'. Chỉ cho phép có (.) và gạch dưới.
2var_name	invalid (không hợp lệ)	Bắt đầu bằng một số
.var_name var.name	valid(hợp lệ)	Có thể bắt đầu bằng (.) nhưng không được theo sau dấu (.) bằng số.

<code>.2var_name</code>	invalid(ko hợp lệ)	Dấu chấm bắt đầu nhưng theo sau là số làm cho nó ko hợp lệ.
<code>_var_name</code>	invalid(ko hợp lệ)	Bắt đầu bằng <code>_</code> không hợp lệ

2.1. Gán biến

Các biến có thể được gán bằng cách sử dụng toán tử sang trái, sang phải và bằng. Giá trị của các biến có thể được in bằng hàm using **print()** hoặc **cat()**. Hàm **cat()** kết hợp nhiều mục thành output in ra liên tục.

```
# Phép gán sử dụng bằng toán tử bằng.
var.1 = c(0,1,2,3)

# Phép gán sử dụng leftward.
var.2 <- c("learn","R")

cat ("var.1 is ", var.1 ,"\n")
cat ("var.2 is ", var.2 ,"\n")
```

Khi thực hiện đoạn code code, Kết quả

```
var.1 is  0 1 2 3
var.2 is  learn R
```

Note – Vector `c(TRUE,1)` có sự kết hợp giữa lớp logic và số. Vì vậy, lớp logic bị ép buộc thành lớp tạo TRUE thành 1.

2.2. Tìm biến

Để biết tất cả các biến có trong không gian làm việc, chúng ta sử dụng hàm **ls()**. Ngoài ra, hàm **ls()** có thể sử dụng mẫu để khớp tên biến.

```
print(ls())
```

Khi thực hiện đoạn code code, Kết quả

```
[1] "my var"      "my_new_var"  "my_var"      "var.1"
[5] "var.2"      "var.3"      "var.name"    "var_name2."
[9] "var_x"      "varname"
```

Note – Đây là output mẫu tùy thuộc vào biến nào được khai báo trong môi trường của bạn.

Hàm `ls()` có thể sử dụng các mẫu để khớp với tên biến.

```
# Liệt kê các biến bằng mẫu "var".
print(ls(pattern = "var"))
```

Khi thực hiện đoạn code code, Kết quả

```
[1] "my var"      "my_new_var" "my_var"      "var.1"
[5] "var.2"      "var.3"      "var.name"    "var_name2."
[9] "var_x"      "varname"
```

Các biến bắt đầu bằng `dot(.)` bị ẩn, chúng có thể được liệt kê bằng cách sử dụng đối số `"all.names = TRUE"` cho hàm `ls()`.

```
print(ls(all.name = TRUE))
```

Khi thực hiện đoạn code code, Kết quả

```
[1] ".cars"          ".Random.seed" ".var_name"      ".varname"
".varname2"
[6] "my var"      "my_new_var"  "my_var"      "var.1"      "var.2"
[11]"var.3"      "var.name"    "var_name2."  "var_x"
```

2.3. Xóa biến

Có thể xóa các biến bằng cách sử dụng hàm `rm()`. Dưới đây chúng ta xóa biến `var.3`. Giá trị in ra của biến là `error`.

```
rm(var.3)
print(var.3)
```

Khi thực hiện đoạn code code, Kết quả

```
[1] "var.3"
Error in print(var.3) : object 'var.3' not found
```

Có thể xóa các biến bằng cách sử dụng đồng thời hàm `rm()` và `ls()`.

```
rm(list = ls())
print(ls())
```

Khi thực hiện đoạn code code, Kết quả

```
character(0)
```

3. Toán tử

3.1. Toán tử số học

Bảng sau hiển thị các toán tử số học được hỗ trợ bởi ngôn ngữ R. Các toán tử tác động lên từng phần tử của vectơ.

Operator	Miêu tả	Example
+	Cộng hai vecto	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v+t)</pre> Kết quả <pre>[1] 10.0 8.5 10.0</pre>
-	Trừ vecto thứ 2 từ vecto thứ nhất	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v-t)</pre> Kết quả <pre>[1] -6.0 2.5 2.0</pre>
*	Nhân hai vecto	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v*t)</pre> Kết quả <pre>[1] 16.0 16.5 24.0</pre>
/	Chia vecto thứ nhất với vecto thứ hai	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v/t)</pre> When we execute the above code, Kết quả <pre>[1] 0.250000 1.833333 1.500000</pre>
%%	Chia lấy phần dư của vectơ thứ nhất cho vecto thứ hai	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v%%t)</pre> Kết quả <pre>[1] 2.0 2.5 2.0</pre>
%%/	Lấy kết quả của vecto thứ nhất cho vecto thứ hai (thương)	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v%%/t)</pre>

		Kết quả [1] 0 1 1
^	Vecto thứ nhất tăng theo số mũ của vecto thứ hai	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v^t)</pre> Kết quả [1] 256.000 166.375 1296.000

3.2. Toán tử quan hệ

Bảng sau hiển thị các toán tử quan hệ được hỗ trợ bởi ngôn ngữ R. Mỗi phần tử của vecto thứ nhất được so sánh với phần tử tương ứng của vecto thứ hai. Kết quả so sánh là một giá trị Boolean.

Operator	Miêu tả	Example
>	Kiểm tra xem mỗi phần tử của vecto đầu tiên có lớn hơn phần tử tương ứng vecto thứ hai hay không.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v>t)</pre> Kết quả [1] FALSE TRUE FALSE FALSE
<	Kiểm tra xem mỗi phần tử của vecto đầu tiên có nhỏ hơn phần tử tương ứng vecto thứ hai hay không.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v < t)</pre> Kết quả [1] TRUE FALSE TRUE FALSE
==	Kiểm tra xem mỗi phần tử của vecto đầu tiên có bằng phần tử tương ứng vecto thứ hai hay không.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v == t)</pre> Kết quả [1] FALSE FALSE FALSE TRUE
<=	Kiểm tra xem mỗi phần tử của vecto đầu tiên	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9)</pre>

	có nhỏ hơn hoặc bằng phần tử tương ứng vecto thứ hai hay không.	<pre>print(v<=t)</pre> Kết quả <pre>[1] TRUE FALSE TRUE TRUE</pre>
>=	Kiểm tra xem mỗi phần tử của vecto đầu tiên có lớn hơn hoặc bằng phần tử tương ứng vecto thứ hai hay không.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v>=t)</pre> Kết quả <pre>[1] FALSE TRUE FALSE TRUE</pre>
!=	Kiểm tra xem mỗi phần tử của vecto đầu tiên có bằng phần tử tương ứng vecto thứ hai hay không	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v!=t)</pre> Kết quả <pre>[1] TRUE TRUE TRUE FALSE</pre>

3.3. Toán tử logic

Bảng sau hiển thị các toán tử logic được hỗ trợ bởi ngôn ngữ R. Nó chỉ áp dụng cho các vecto loại logic, số hoặc phức tạp. Tất cả các số lớn hơn 1 được coi là giá trị logic TRUE.

Mỗi phần tử của vecto thứ nhất được so sánh với phần tử tương ứng của vecto thứ hai, kết quả so sánh là 1 giá trị Boolean.

Operator	Miêu tả	Example
&	It is called Element-wise Logical AND operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if both the elements are TRUE.	<pre>v <- c(3,1,TRUE,2+3i) t <- c(4,1,FALSE,2+3i) print(v&t)</pre> Kết quả <pre>[1] TRUE TRUE FALSE TRUE</pre>

	It is called Element-wise Logical OR operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if one the elements is TRUE.	<pre>v <- c(3,0,TRUE,2+2i) t <- c(4,0,FALSE,2+3i) print(v t)</pre> Kết quả <pre>[1] TRUE FALSE TRUE TRUE</pre>
!	It is called Logical NOT operator. Takes each element of the vector and gives the opposite logical value.	<pre>v <- c(3,0,TRUE,2+2i) print(!v)</pre> Kết quả <pre>[1] FALSE TRUE FALSE FALSE</pre>

The logical operator && and || considers only the first element of the vectors and give a vector of single element as output.

Operator	Description	Example
&&	Called Logical AND operator. Takes first element of both the vectors and gives the TRUE only if both are TRUE.	<pre>v <- c(3,0,TRUE,2+2i) t <- c(1,3,TRUE,2+3i) print(v&& t)</pre> Kết quả <pre>[1] TRUE</pre>
	Called Logical OR operator. Takes first element of both the vectors and gives the TRUE if one of them is TRUE.	<pre>v <- c(0,0,TRUE,2+2i) t <- c(0,3,TRUE,2+3i) print(v t)</pre> Kết quả <pre>[1] FALSE</pre>

3.4. Assignment Operators

These operators are used to assign values to vectors.

Operator	Description	Example
<code><-</code> or <code>=</code> or <code><<-</code>	Called Left Assignment	<pre>v1 <- c(3,1,TRUE,2+3i) v2 <<- c(3,1,TRUE,2+3i) v3 = c(3,1,TRUE,2+3i) print(v1) print(v2) print(v3)</pre> Kết quả <pre>[1] 3+0i 1+0i 1+0i 2+3i [1] 3+0i 1+0i 1+0i 2+3i [1] 3+0i 1+0i 1+0i 2+3i</pre>
<code>-></code> or <code>->></code>	Called Right Assignment	<pre>c(3,1,TRUE,2+3i) -> v1 c(3,1,TRUE,2+3i) ->> v2 print(v1) print(v2)</pre> Kết quả <pre>[1] 3+0i 1+0i 1+0i 2+3i [1] 3+0i 1+0i 1+0i 2+3i</pre>

3.5. Miscellaneous Operators

These operators are used to for specific purpose and not general mathematical or logical computation.

Operator	Description	Example
:	Colon operator. It creates the series of numbers in sequence for a vector.	<pre>v <- 2:8 print(v)</pre> Kết quả <pre>[1] 2 3 4 5 6 7 8</pre>
<code>%in%</code>	This operator is used to identify if an	<pre>v1 <- 8 v2 <- 12 t <- 1:10 print(v1 %in% t)</pre>

	element belongs to a vector.	<pre>print(v2 %in% t)</pre> Kết quả <pre>[1] TRUE [1] FALSE</pre>
%%	This operator is used to multiply a matrix with its transpose.	<pre>M = matrix(c(2,6,5,1,10,4), nrow = 2, ncol = 3, byrow = TRUE) t = M %% t(M) print(t)</pre> Kết quả <pre> [,1] [,2] [1,] 65 82 [2,] 82 117</pre>

4. Decision making

R provides the following types of decision making statements

4.1. If-else statement

4.1.1. Cú pháp

The basic syntax for creating an If-else statement in R is -

```
if(boolean_expression 1) {
  // Executes when the boolean expression 1 is true.
} else if( boolean_expression 2) {
  // Executes when the boolean expression 2 is true.
} else if( boolean_expression 3) {
  // Executes when the boolean expression 3 is true.
} else {
  // executes when none of the above condition is true.
}
```

4.1.2. Ví dụ

```
x <- c("what","is","truth")

if("Truth" %in% x) {
  print("Truth is found the first time")
} else if ("truth" %in% x) {
  print("truth is found the second time")
} else {
  print("No truth found")
}
```

When the above code is compiled and executed, Kết quả

```
[1] "X is an Integer"
```

4.2. Switch Statement

4.2.1. Cú pháp

The basic syntax for creating a switch statement in R is -

```
switch(expression, case1, case2, case3....)
```

4.2.2. Ví dụ

```
x <- switch(
  3,
  "first",
  "second",
  "third",
  "fourth"
)
print(x)
```

When the above code is compiled and executed, Kết quả

```
[1] "third"
```

5. Vòng lặp

5.1. For Loop

5.1.1. Syntax

The basic syntax for creating a **for** loop statement in R is -

```
for (value in vector) {
  statements
}
```

5.1.2. Example

Example 1:

```
v <- LETTERS[1:4]
for ( i in v) {
  print(i)
}
```

When the above code is compiled and executed, Kết quả

```
[1] "A"
[1] "B"
[1] "C"
[1] "D"
```

Example 1:

```
for ( i in 1:5) {  
  print(i)  
}
```

When the above code is compiled and executed, Kết quả

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

5.2. While Loop

5.2.1. Cú pháp

The basic syntax for creating a while loop in R is -

```
while (test_expression) {  
  statement  
}
```

5.2.2. Ví dụ

```
v <- c("Hello","while loop")  
cnt <- 0  
  
while (cnt < 3) {  
  print(v)  
  cnt = cnt + 1  
}
```

When the above code is compiled and executed, Kết quả

```
[1] "Hello" "while loop"  
[1] "Hello" "while loop"  
[1] "Hello" "while loop"
```

5.3. Loop Control Statements

R supports the following control statements.

5.3.1. Break

```
v <- c("Hello","loop")  
cnt <- 2  
  
repeat {  
  print(v)
```

```
    cnt <- cnt + 1

    if(cnt > 5) {
        break
    }
}
```

When the above code is compiled and executed, Kết quả

```
[1] "Hello" "loop"
[1] "Hello" "loop"
[1] "Hello" "loop"
[1] "Hello" "loop"
```

5.3.2. Next

```
v <- LETTERS[1:6]
for ( i in v) {

    if (i == "D") {
        next
    }
    print(i)
}
```

When the above code is compiled and executed, Kết quả

```
[1] "A"
[1] "B"
[1] "C"
[1] "E"
[1] "F"
```

6. Function

6.1. User-defined and Calling a Function

```
# Create a function to print squares of numbers in sequence.
new.function <- function(a) {
    for(i in 1:a) {
        b <- i^2
        print(b)
    }
}

# Call the function new.function supplying 6 as an argument.
new.function(6)
```

6.2. Một vài hàm cơ bản

6.2.1. Toán học

Function	Biểu thức Đại số	Ví dụ
abs(x)	$ x $	<pre>v <- abs(-10) print(v)</pre> Kết quả <pre>[1] 10</pre>
sin(x), cos(x), tan(x)	Sin(x), cos(x), tan(x)	<pre>v <- sin(pi/2) print(v)</pre> Kết quả <pre>[1] 1</pre>
log(a,b), log(x) log10(y) exp(c)	$\text{Log}_b(a)$, $\ln(x)$, $\log_{10}(y)$, e^c	<pre>v <- log10(10) print(v)</pre> Kết quả <pre>[1] 1</pre>

6.2.2. Tính toán ma trận

Function	Mô tả	Ví dụ
t()	Chuyển vị ma trận	<pre>M <- matrix(1:4,nrow = 2) print(M) print(t(M))</pre> Kết quả <pre> [,1] [,2] [1,] 1 3 [2,] 2 4 [,1] [,2] [1,] 1 2 [2,] 3 4</pre>
det()	Tính định thức	<pre>M <- matrix(1:4,nrow = 2) print(det(M))</pre>

		Kết quả <pre>[1] -2</pre>
solve()	Tính ma trận nghịch đảo	<pre>M <- matrix(1:4,nrow = 2) print(solve(M))</pre> Kết quả <pre> [,1] [,2] [1,] -2 1.5 [2,] 1 -0.5</pre>

6.2.3. Thống kê mô tả

Function	Mô tả
Trung bình	mean()
Trung vị	median()
Phương sai	var()
Độ lệch chuẩn	sd()
Phân bố chuẩn	rnorm()
Phân bố nhị thức	rbinom()
Phân bố Poisson	rpois()
Phân bố đều	runif()

6.2.4. Khác

Function	Mô tả
Tổng	sum()
Tích	prod()
Số phần tử	length()
Sắp xếp	sort()
Tạo dãy số từ a đến b cách x	seq(a,b,x)
Tạo một dãy số lặp	<pre>rep() > rep(1:3, times = 3) [1] 1 2 3 1 2 3 1 2 3 > rep(1:3, times = c(3, 3, 3)) [1] 1 1 1 2 2 2 3 3 3</pre>

7. Dữ liệu

7.1. Nhập dữ liệu

Cách nhập	Mô tả	Ví dụ
-----------	-------	-------

<code>c(); rep()</code>	Nhập trực tiếp	<code>age <- c(18,20,19,22)</code>
<code>read.csv()</code>	Đọc dữ liệu từ file csv	<code>data <- read.csv("WH01.csv", header = TRUE)</code>
<code>read_excel()</code>	Đọc từ file Excel	<code>library(readxl)</code> <code>ex<- readxl_example("datasets.xlsx")</code> <code>read_excel(ex)</code>
<code>data()</code>	Đọc dữ liệu từ packages	<code>library(MASS)</code> <code>data(Boston)</code>

7.2. Xử lý dữ liệu

Trong quá trình xử lý dữ liệu, ta thường gặp 3 loại vector: vector số, factor, vector chữ. Để chuyển đổi giữa 3 loại vector này, ta sử dụng các hàm sau:

`as.factor()` : Để chuyển thành factor

`as.character()` : Để chuyển thành vector chữ.

`as.numeric()` : Để chuyển thành vector số.

7.3. Thao tác với dữ liệu

Trong suốt phần này ta sẽ sử dụng dữ liệu “df=mtcars”

```
df <- mtcars
```

```
df$trans <- factor(df$am, levels=c(0, 1), labels=c("Automatic", "Manual"))
```

7.3.1. Xử lý dữ liệu bị thiếu

* Để xóa bỏ các hàng dữ liệu bị thiếu, chúng ta sử dụng hàm `na.omit()`

```
data <- read.csv("melb_data.csv")
data2 <- na.omit(data)
```

* Để thống kê số lượng ô bị thiếu dữ liệu ở từng cột, chúng ta sử dụng hàm `is.na()` và hàm `colSums()` .

```
data <- read.csv("melb_data.csv")
colSums(is.na(data))
```

* Để thay thế dữ liệu định lượng bị thiếu bằng các giá trị khác (ví dụ: giá trị trung bình), ta sử dụng các hàm của R để làm việc này.

```
a <- read.csv("melb_data.csv")
colSums(is.na(data))
a[is.na(data$BuildingArea), ]$BuildingArea <-
mean(data$BuildingArea, na.rm=TRUE)
colSums(is.na(data))
```

7.3.2. Trích dữ liệu

* Trích dữ liệu bằng lệnh Subset

```
# trích dữ liệu con gồm các hàng có am = 0
> Automatic = subset(df, df$am == 0)
# trích dữ liệu con gồm các hàng có am = 1
> Manual = subset(df, df$am == 1)
> dim(Manual)
```

```
[1] 13 11
```

```
> Manual3 = subset(df, df$am == 1 & df$wt > 2)
> dim(Manual3)
```

```
[1] 9 11
```

* Sử dụng R's built-in function

```
# Trích véc tơ hàng 3 của dữ liệu df
> data1 = df[3, ]

# Trích véc tơ cột 3 của dữ liệu df
> data2 = df[, 3]

# Trích dữ liệu con gồm hàng 1, 2, 3 và cột 4, 5, 6 của dữ liệu df.
> data4 = df[c(1, 2, 3), c(4, 5, 6)]

# trích dữ liệu con gồm các hàng có am = 1
> Manual = df[df$am == 1]
> dim(Manual3)
```

```
[1] 9 11
```

7.4. Họ các hàm apply

7.4.1. apply

- Cấu trúc:

```
apply(X, MARGIN, FUN)
```

- Trong đó:

X: là một vector hoặc matrix

MARGIN: Nếu MARGIN = 1 thì sẽ áp dụng với hàng, MARGIN = 2 với cột
FUN: tên của hàm mà ta muốn áp dụng, có thể tự định nghĩa.

- Ví dụ:

```
sample_matrix <- matrix(1:10, nrow=3, ncol=10)
print("sum across rows:")
apply( sample_matrix, 1, sum)
```

7.4.2. lapply() và sapply()

- Cấu trúc:

```
lapply(X, FUN)
sapply(X, FUN)
```

Với các tham số tương tự như hàm apply()

Lưu ý: rằng hàm lapply() luôn trả về kết quả là 1 list và sapply trả về vector hoặc matrix.

8. Trình bày và mô tả dữ liệu

8.1. Các khái niệm cơ bản

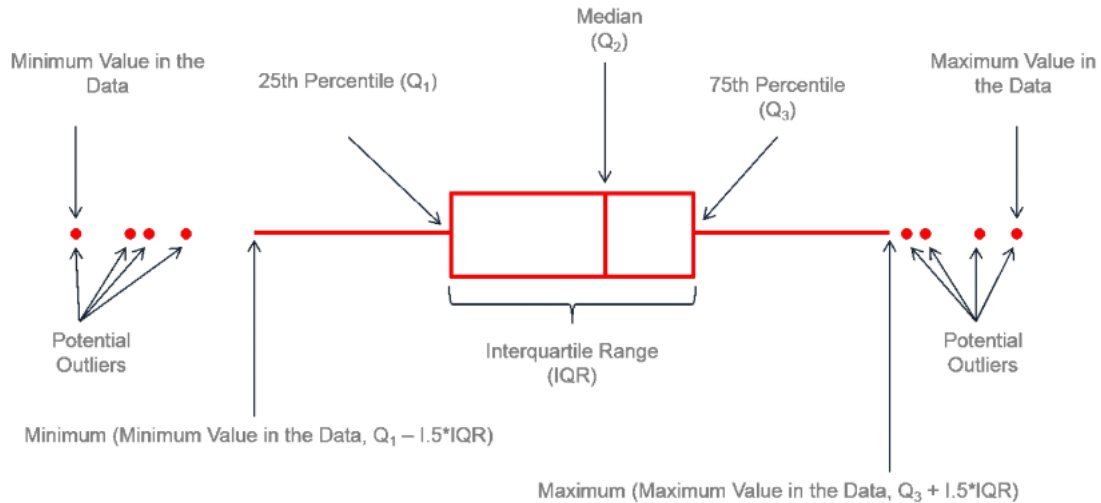
8.1.1. Các số đo xu thế trung tâm

- + Trung bình mẫu: Là trung bình cộng các giá trị ta quan sát được.
- + Trung vị mẫu: Là số m có tính chất: Số các giá trị của mẫu bé hơn
hay bằng m thì bằng số giá trị của mẫu lớn hơn hay bằng m
- + Mode: Là giá trị trong mẫu xuất hiện với tần số lớn nhất.

8.1.2. Các số đo mức độ phân tán

- + Biên độ mẫu: Là hiệu số giữa giá trị lớn nhất và giá trị bé nhất của mẫu.
- + Khoảng tứ phân vị: Là hiệu số $Q_3 - Q_1$, trong đó Q_1 là giá trị mà 25% số liệu nhỏ hơn Q_1 , Q_3 là giá trị mà 75% số liệu nhỏ hơn Q_3
- + Độ lệch trung bình mẫu: Là giá trị trung bình của khoảng cách từ mỗi giá trị đến trung bình mẫu.
- + Phương sai: Là giá trị trung bình của bình phương khoảng cách từ mỗi giá trị đến trung bình mẫu.
- + Độ lệch chuẩn mẫu: là căn bậc hai của phương sai.

8.2. Phát hiện các giá trị điểm bất thường



9. Trình bày dữ liệu

9.1. Một số khái niệm

9.1.1. Các phương pháp biểu diễn dữ liệu:

+ Sử dụng bảng phân bố:

+ Bảng phân bố tần số: là bảng dùng để biểu diễn một tập dữ liệu cho biết các giá trị xuất hiện và số lần xuất hiện mỗi giá trị đó

+ Bảng phân bố tần số ghép lớp: là bảng biểu diễn dữ liệu dạng khoảng, dữ liệu định tính, . . . theo đó các giá trị được ghép thành các lớp (các nhóm) cùng với số giá trị thuộc mỗi lớp (tần số).

+ Biểu đồ:

+ Biểu đồ hình cột: Dùng để đưa ra so sánh giữa các nhóm.

+ Biểu đồ hình quạt: Dùng để phân tích hoặc so sánh ở mức độ tổng thể

+ Tổ chức đồ (histogram): Dùng để biểu thị tần số/ tần suất các giá trị trong mỗi khoảng giá trị.

+ Biểu đồ tán xạ: Dùng để quan sát mối liên hệ giữa hai biến.

9.1.2. Các số đo xu thế trung tâm:

+ Trung bình mẫu: Là trung bình cộng các giá trị ta quan sát được.

+ Trung vị mẫu: Là số m có tính chất: Số các giá trị của mẫu bé hơn hay bằng m thì bằng số giá trị của

mẫu lớn hơn hay bằng m

+ Mode: Là giá trị trong mẫu xuất hiện với tần số lớn nhất.

9.1.3. Các số đo mức độ phân tán:

+ Biên độ mẫu: Là hiệu số giữa giá trị lớn nhất và giá trị bé nhất của mẫu.

+ Khoảng tứ phân vị: Là hiệu số $Q_3 - Q_1$, trong đó Q_1 là giá trị mà 25% số liệu nhỏ hơn Q_1 , Q_3 là giá trị mà 75% số liệu nhỏ hơn Q_3

+ Độ lệch trung bình mẫu: Là giá trị trung bình của khoảng cách từ mỗi giá trị đến trung bình mẫu.

+ Phương sai: Là giá trị trung bình của bình phương khoảng cách từ mỗi giá trị đến trung bình mẫu.

+ Độ lệch chuẩn mẫu: là căn bậc hai của phương sai.

9.2. Biểu đồ cột

9.2.1. Cách vẽ

* Cú pháp

Cú pháp cơ bản của biểu đồ cột trong R là

```
barplot(H,xlab,ylab,main, names.arg,col)
```

Với các thông số được miêu tả chi tiết sau

- **H** là một véc-tơ hoặc một ma trận chứa các giá trị cần biểu diễn
- **xlab** là đơn vị hoặc mô tả cho trục Ox
- **ylab** là đơn vị hoặc mô tả cho trục Oy
- **main** đặt tên cho biểu đồ cột
- **names.arg** là một véc-tơ chứa tên hoặc mô tả nằm dưới từng cột tương ứng
- **col** chọn màu cho các cột

* Ví dụ

Một biểu đồ cột đơn giản với một véc-tơ dữ liệu đầu vào và tên của từng cột

```
# Create the data for the chart
H <- c(7,12,28,3,41)

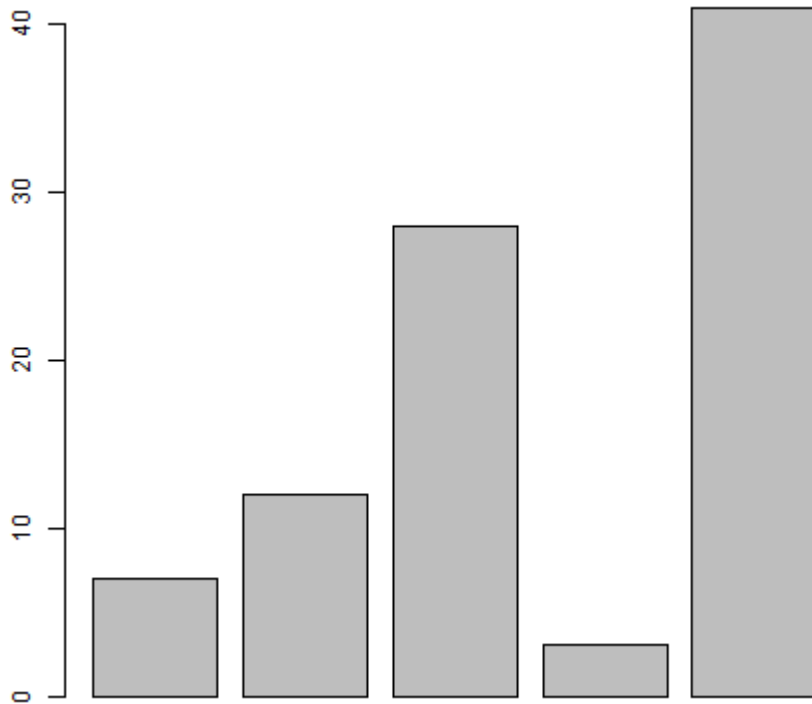
# Give the chart file a name
png(file = "barchart.png")

# Plot the bar chart
```

```
barplot(H)
```

```
# Save the file  
dev.off()
```

Khi thực thi đoạn code trên, ta có kết quả là biểu đồ sau:

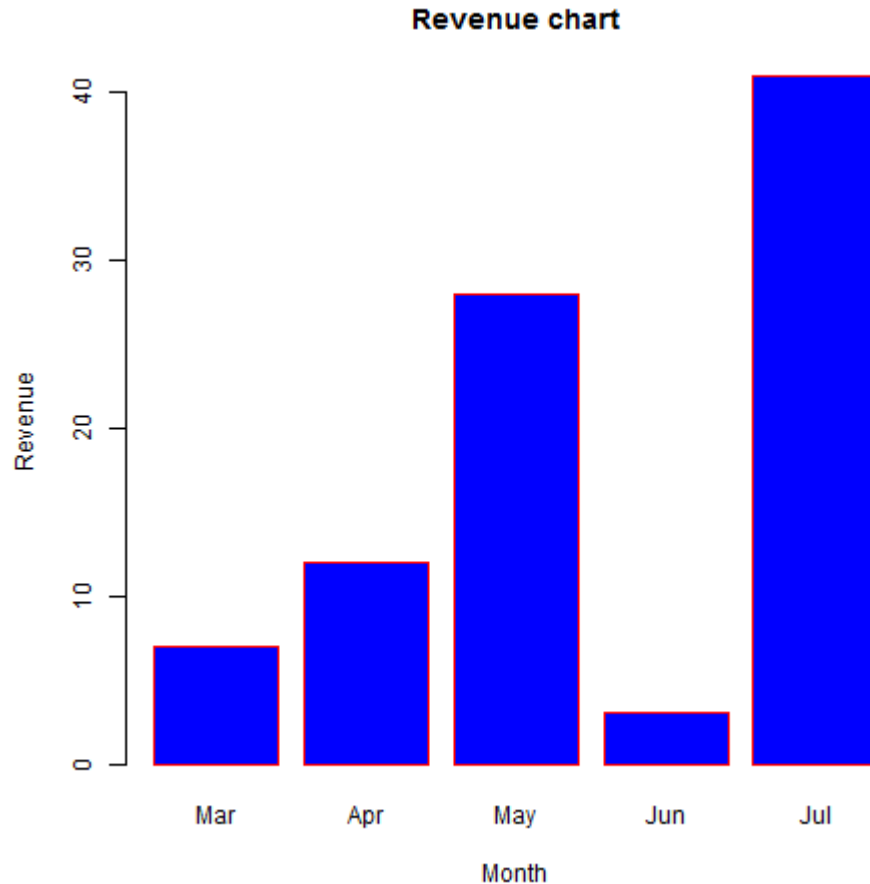


9.2.2. Biểu đồ cột khi thêm thông số labels, title và colors

* Ví dụ

```
# Create the data for the chart  
H <- c(7,12,28,3,41)  
M <- c("Mar","Apr","May","Jun","Jul")  
  
# Give the chart file a name  
png(file = "barchart_months_revenue.png")  
  
# Plot the bar chart  
barplot(H,names.arg=M,xlab="Month",ylab="Revenue",col="blue",  
main="Revenue chart",border="red")  
  
# Save the file  
dev.off()
```

Khi thực thi đoạn code trên, ta có kết quả là biểu đồ sau:



9.3. Biểu đồ cột kép và biểu đồ cột chồng

Chúng ta có thể tạo ra biểu đồ cột với groups of bars và stacks trong từng cột bằng cách cho dữ liệu đầu vào là một ma trận. Nhiều hơn 2 biến được cho trong ma trận (biểu hiện ở số hàng).

```
# Create the input vectors.
colors = c("green","orange","brown")
months <- c("Mar","Apr","May","Jun","Jul")
regions <- c("East","West","North")

# Create the matrix of the values.
Values <- matrix(c(2,9,3,11,9,4,8,7,3,12,5,2,8,10,11), nrow = 3,
ncol = 5, byrow = TRUE)

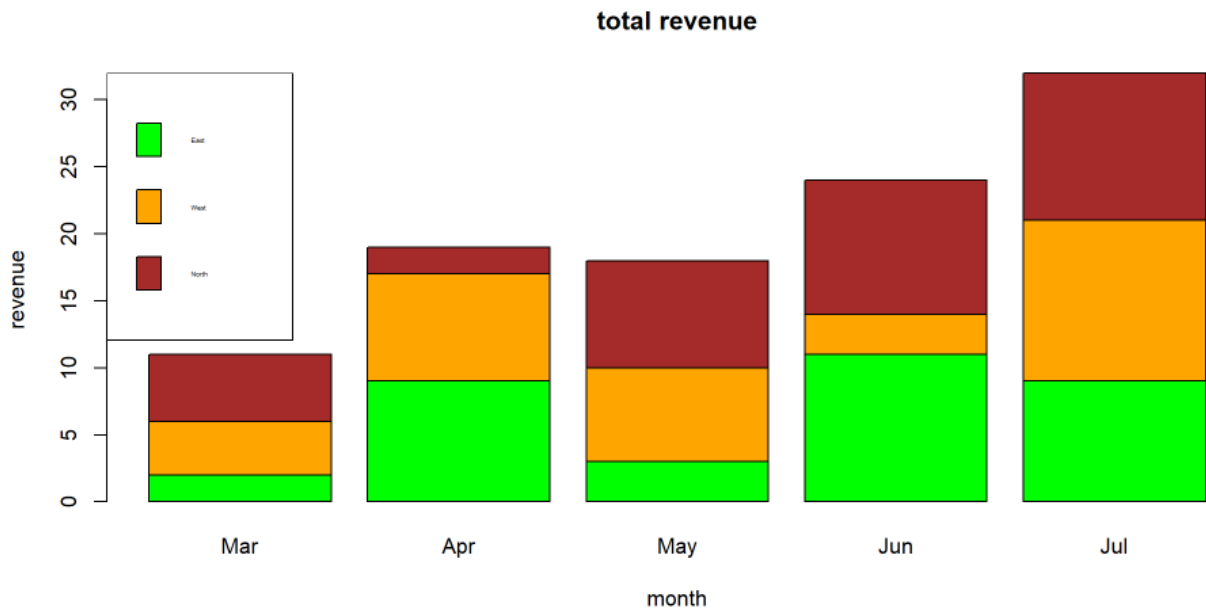
# Give the chart file a name
png(file = "barchart_stacked.png")
```

```
# Create the bar chart
barplot(Values, main = "total revenue", names.arg = months, xlab =
"month", ylab = "revenue", col = colors)

# Add the legend to the chart
legend("topleft", regions, cex = 0.3, fill = colors)

# Save the file
dev.off()
```

Khi thực thi đoạn code trên, ta có kết quả là biểu đồ sau:



9.4. Histograms

9.4.1. Cách vẽ

Biểu đồ histogram dùng để biểu thị tần số của các giá trị của một biến thành các dãy giá trị. Biểu đồ histogram tương tự như các thanh bar trong biểu đồ cột, nhưng điểm khác biệt là nó nhóm các giá trị thành các dãy thanh liên tục. Mỗi thanh trong biểu đồ biểu thị chiều cao của số lượng giá trị có trong phạm vi đó.

R tạo biểu đồ bằng hàm **hist()**, trong đó một véc-tơ làm dữ liệu đầu vào và sử dụng thêm một số tham số để vẽ biểu đồ.

* Cú pháp

Cú pháp cơ bản để tạo biểu đồ histogram trong R là

```
hist(v,main,xlab,xlim,ylim,breaks,col,border)
```

Với miêu tả thành phần chi tiết là

- **v** là véc-tơ đầu vào chứa dãy giá trị cần biểu diễn
- **main** đặt tiêu đề cho biểu đồ
- **col** chọn màu cho các thanh bar.
- **border** chọn màu viền bên ngoài các thanh bar.
- **xlab** miêu tả thông tin của giá trị trên trục Ox
- **xlim** đặt khoảng giá trị giới hạn trên trục Ox (tức ngoài khoảng đó ra thì giá trị đầu vào ở v không nằm trong khoảng đó không hiện trong biểu đồ)
- **ylim** tương tự với xlim nhưng trên trục Oy
- **breaks** đặt chiều rộng của mỗi thanh bar

* Example

Một biểu đồ histogram đơn giản với thông số đầu vào gồm **véc-tơ**, **label**, **col** và **border**.

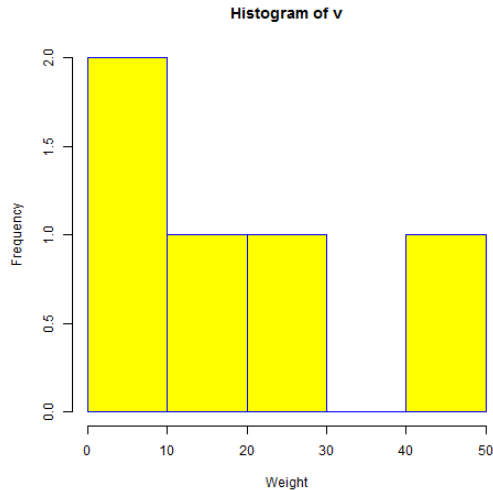
```
# Create data for the graph.
v <- c(9,13,21,8,36,22,12,41,31,33,19)

# Give the chart file a name.
png(file = "histogram.png")

# Create the histogram.
hist(v,xlab = "Weight",col = "yellow",border = "blue")

# Save the file.
dev.off()
```

Khi thực thi đoạn code, ta được kết quả là biểu đồ dưới đây:



9.4.2. Biểu đồ khi cho giới hạn giá trị X và Y

Để chỉ định rõ khoảng giới hạn giá trị trên trục Ox và Oy , ta dùng thông số `xlim` và `ylim`.

Có thể set chiều rộng của mỗi thanh bar bằng cách thêm thông số `breaks`.

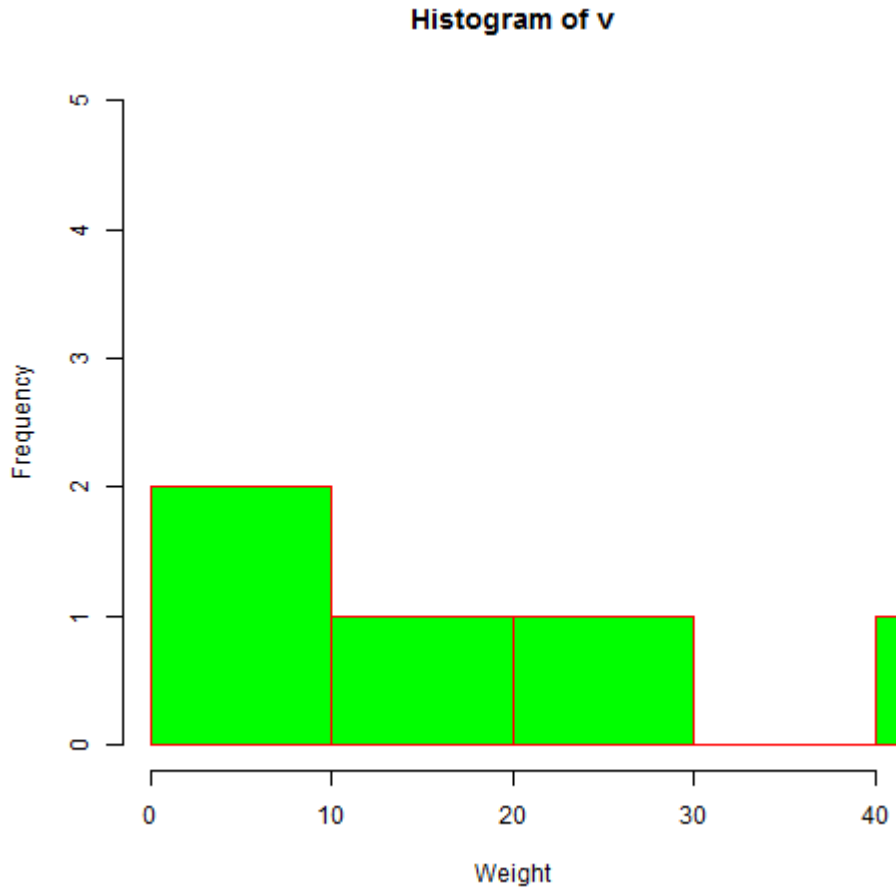
```
# Create data for the graph.
v <- c(9,13,21,8,36,22,12,41,31,33,19)

# Give the chart file a name.
png(file = "histogram_lim_breaks.png")

# Create the histogram.
hist(v,xlab = "Weight",col = "green",border = "red", xlim =
c(0,40), ylim = c(0,5),
    breaks = 5)

# Save the file.
dev.off()
```

Khi thực thi đoạn code trên, ta có kết quả là biểu đồ sau:



9.5. Biểu đồ đường

9.5.1. Cách vẽ

* Cú pháp

Cú pháp cơ bản của biểu đồ đường trong R là:

```
plot(v,type,col,xlab,ylab)
```

Với các thông số chi tiết miêu tả là

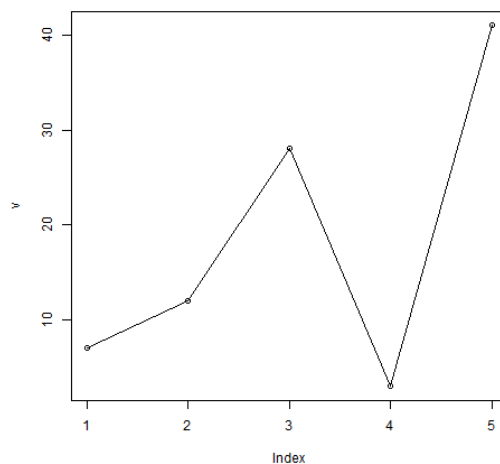
- **v** là vectơ chứa các dãy giá trị cần biểu diễn
- **type** chọn giá trị “p” để biểu diễn các điểm giá trị, “l” để vẽ mỗi các đường giá trị và chọn “o” để biểu diễn cả điểm và đường.
- **xlab** cho biết đơn vị của trục Ox
- **ylab** cho biết đơn vị của trục Oy
- **main** đặt tiêu đề của biểu đồ
- **col** chọn màu biểu diễn cho điểm và đường vẽ

* Ví dụ

Một biểu đồ đường đơn giản với mỗi véc-tơ dữ liệu đầu vào và type = "o".

```
# Create the data for the chart.  
v <- c(7,12,28,3,41)  
  
# Give the chart file a name.  
png(file = "line_chart.jpg")  
  
# Plot the bar chart.  
plot(v,type = "o")  
  
# Save the file.  
dev.off()
```

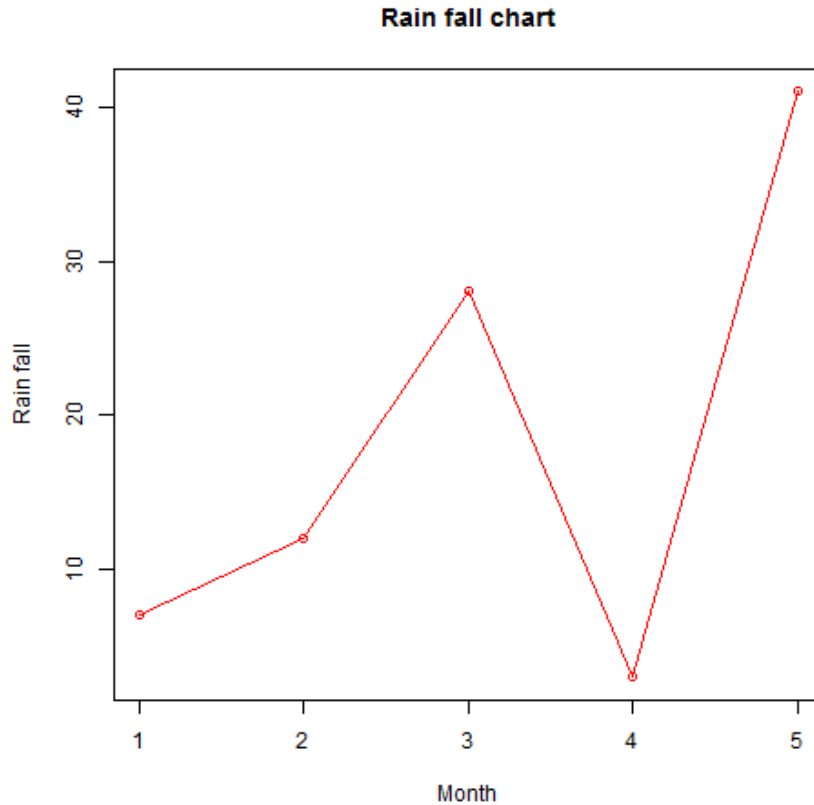
Khi thực thi đoạn code trên, ta được kết quả là biểu đồ sau:



9.5.2. Hàm biểu đồ đường với các thông số title, color và labels

* Ví dụ

```
# Create the data for the chart.  
v <- c(7,12,28,3,41)  
  
# Give the chart file a name.  
png(file = "line_chart_label_colored.jpg")  
  
# Plot the bar chart.  
plot(v,type = "o", col = "red", xlab = "Month", ylab = "Rain fall",  
      main = "Rain fall chart")
```



```
# Save the file.  
dev.off()
```

9.5.3. Biểu đồ chứa nhiều đường

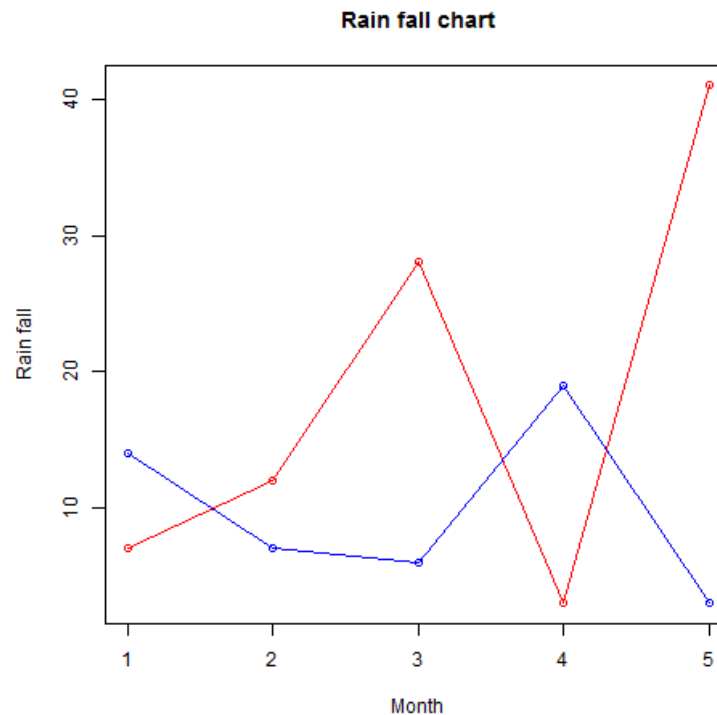
Khi biểu diễn nhiều hơn một đường trong cùng một biểu đồ thì vẫn sử dụng hàm **lines()**.

Sau khi đường đầu tiên được plot vào thì dùng hàm **lines()** để vẽ tiếp đường vectơ thứ 2.

```
# Create the data for the chart.  
v <- c(7,12,28,3,41)  
t <- c(14,7,6,19,3)  
  
# Give the chart file a name.  
png(file = "line_chart_2_lines.jpg")  
  
# Plot the bar chart.  
plot(v,type = "o",col = "red", xlab = "Month", ylab = "Rain fall",  
     main = "Rain fall chart")
```

```
lines(t, type = "o", col = "blue")  
  
# Save the file.  
dev.off()
```

Khi thực thi đoạn code, ta được kết quả là biểu đồ dưới đây:



9.6. Biểu đồ tròn

9.6.1. Cách vẽ

* Cú pháp

Cú pháp cơ bản để tạo biểu đồ tròn pie-chart trong R là:

```
pie(x, labels, radius, main, col, clockwise)
```

Với các thông số chi tiết bên trong

- **x** là vectơ chứa số liệu các thành phần dùng trong biểu đồ
- **labels** là véc-tơ tên hoặc miêu tả các thành phần trong lát cắt
- **radius** cho giá trị bán kính biểu đồ hình tròn (giá trị nằm trong khoảng -1 và +1)
- **main** đặt title cho biểu đồ
- **col** vectơ bảng màu cho các thành phần
- **clockwise** is a logical value indicating if the slices are drawn clockwise or anti clockwise. giá trị điều kiện nhằm

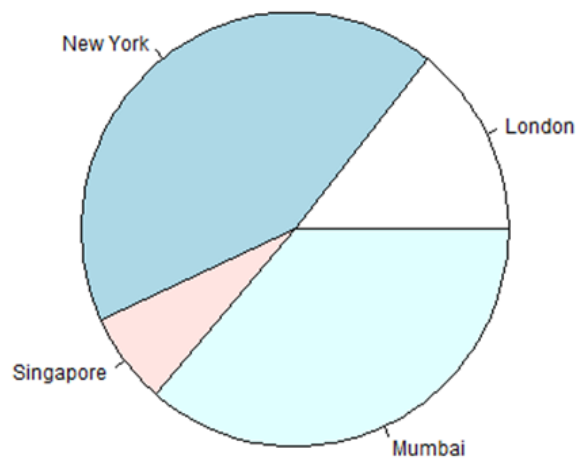
đặt cách vẽ theo chiều kim đồng hồ hoặc ngược chiều kim đồng hồ

* Ví dụ

Một ví dụ đơn giản cho biểu đồ tròn pie chart là cho dữ liệu đầu vào là một véc-tơ x và véc-tơ labels.

```
# Create data for the graph.  
x <- c(21, 62, 10, 53)  
labels <- c("London", "New York", "Singapore", "Mumbai")  
  
# Give the chart file a name.  
png(file = "city.png")  
  
# Plot the chart.  
pie(x, labels)  
  
# Save the file.  
dev.off()
```

Khi chúng ta thực thi đoạn code trên, ta có kết quả là biểu đồ dưới đây



9.6.2. Pie Chart Title and Colors

Có thể mở rộng các tính năng của biểu đồ bằng cách thêm nhiều tham số hơn trong hàm. Chúng ta sẽ sử dụng tham số **main** để thêm tiêu đề vào biểu đồ và một tham số khác là **col** sẽ sử dụng bảng màu cầu vồng

trong khi vẽ biểu đồ. Chiều dài của pallet phải giống với số lượng giá trị chúng ta có cho biểu đồ, `length(x)`.

*** Ví dụ**

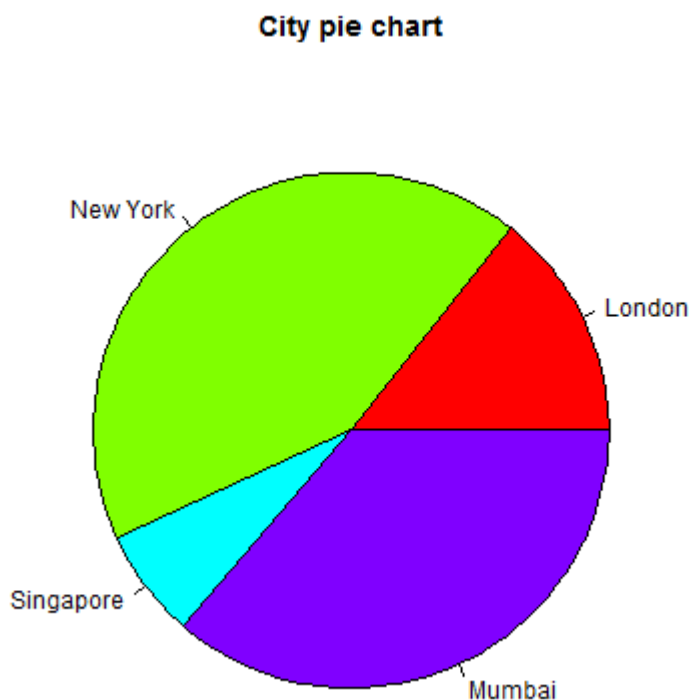
```
# Create data for the graph.
x <- c(21, 62, 10, 53)
labels <- c("London", "New York", "Singapore", "Mumbai")

# Give the chart file a name.
png(file = "city_title_colours.jpg")

# Plot the chart with title and rainbow color pallet.
pie(x, labels, main = "City pie chart", col = rainbow(length(x)))

# Save the file.
dev.off()
```

Khi chúng ta thực thi đoạn code trên, ta có kết quả



9.6.3. Slice Percentages and Chart Legend

Chúng ta cũng có thể thêm thành phần tỷ lệ lát cắt (`piepercent`) và chú giải biểu đồ (`chart legend`) vào trong hàm.

```
# Create data for the graph.
```

```

x <- c(21, 62, 10, 53)
labels <- c("London", "New York", "Singapore", "Mumbai")

piepercent <- round(100*x/sum(x), 1)

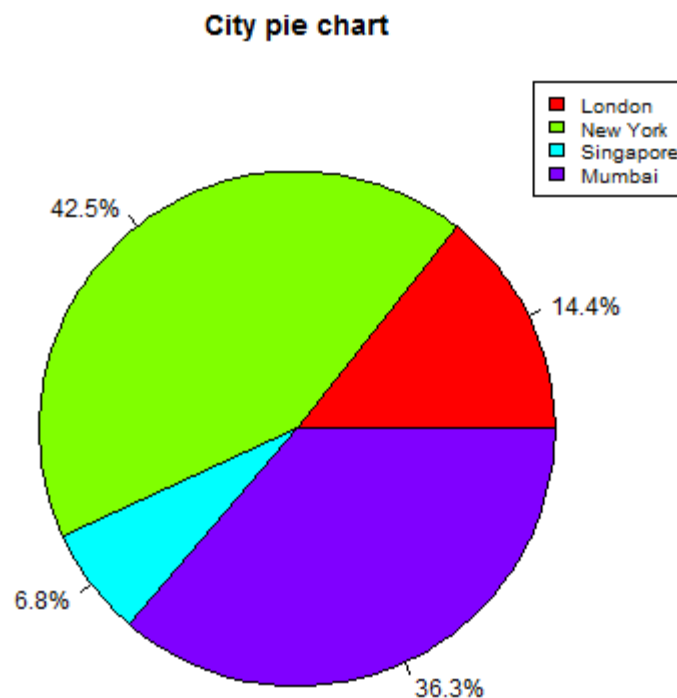
# Give the chart file a name.
png(file = "city_percentage_legends.jpg")

# Plot the chart.
pie(x, labels = piepercent, main = "City pie chart", col =
rainbow(length(x)))
legend("topright", c("London", "New York", "Singapore", "Mumbai"), cex
= 0.8,
      fill = rainbow(length(x)))

# Save the file.
dev.off()

```

Khi thực thi đoạn code trên, ta có kết quả là biểu đồ dưới đây:



9.6.4. 3D Pie Chart

Một biểu đồ pie chart 3 chiều vẽ được khi cài đặt thêm packages bổ sung **plotrix**, nó có hàm **pie3D()** thực hiện lệnh vẽ này.

```
# Get the library.
library(plotrix)

# Create data for the graph.
x <- c(21, 62, 10, 53)
lbl <- c("London", "New York", "Singapore", "Mumbai")

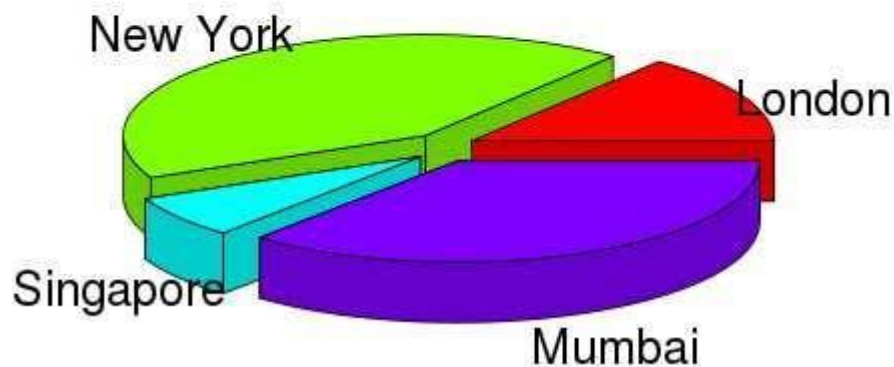
# Give the chart file a name.
png(file = "3d_pie_chart.jpg")

# Plot the chart.
pie3D(x, labels = lbl, explode = 0.1, main = "Pie Chart of Countries
")

# Save the file.
dev.off()
```

Khi thực thi đoạn code trên, ta có kết quả là biểu đồ dưới đây

Pie Chart of Countries



9.7. Biểu đồ Boxplot

9.7.1. Cách vẽ

* Cú pháp

Cú pháp đơn giản để tạo boxplot trong R là

```
boxplot(x, data, notch, varwidth, names, main)
```

Chi tiết các thông số được sử dụng là:

- **x** là 1 véc-tơ biến số
- **data** phải dưới dạng data.frame
- **notch** mặc định là FALSE, set notch = TRUE để tạo notch (để làm gì thì xem phía dưới)
- **varwidth** mặc định là FALSE, set varwidth = TRUE để vẽ hộp tỷ lệ cân xứng với véc-tơ chiều rộng
- **names** đặt tên để dán nhãn từng nhóm dữ liệu, được hiện ở dưới từng boxplot
- **main** đặt title cho biểu đồ

* Ví dụ

Sử dụng data set "mtcars" có sẵn trong môi trường R để tạo một boxplot cơ bản. Hãy xem các cột "mpg" và "cyl" trong mtcars.

```
input <- mtcars[,c('mpg','cyl')]
print(head(input))
```

Khi chúng ta thực thi đoạn code trên thì có bảng data sau:

	mpg	cyl
Mazda RX4	21.0	6
Mazda RX4 Wag	21.0	6
Datsun 710	22.8	4
Hornet 4 Drive	21.4	6
Hornet Sportabout	18.7	8
Valiant	18.1	6

Tạo biểu đồ Boxplot cơ bản

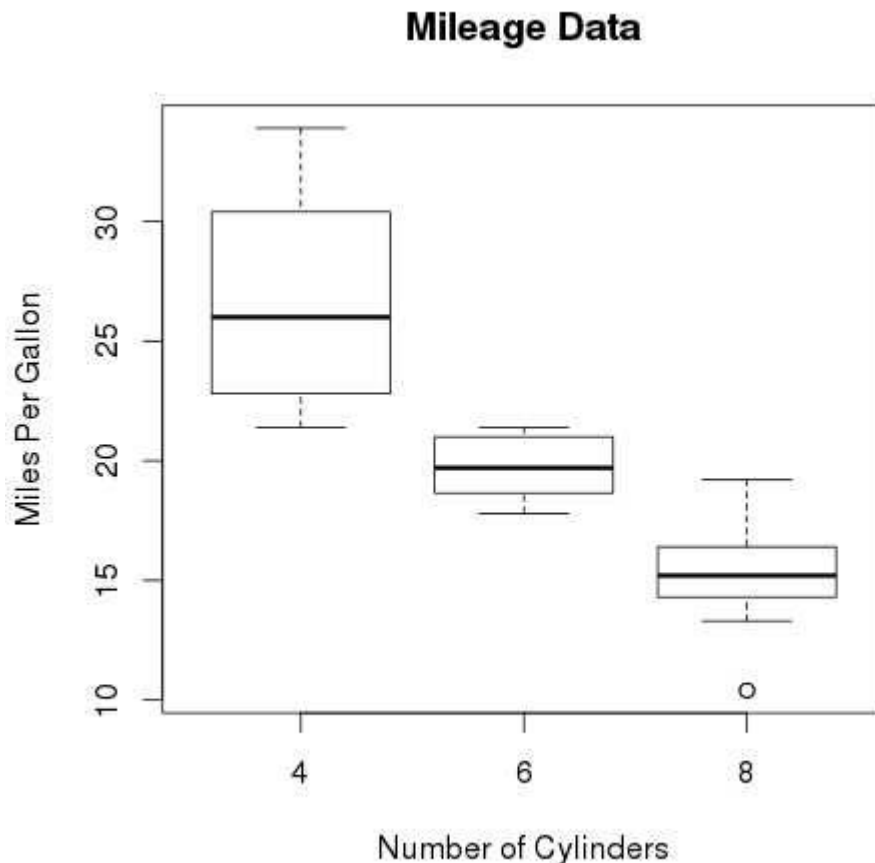
Lệnh dưới đây để tạo một biểu đồ boxplot thể hiện mối tương quan giữa mpg (dặm trên mỗi gallon - đơn vị đo lường mức độ tiêu thụ nhiên liệu oto của Mỹ) và cyl (số lượng xy lanh)

```
# Give the chart file a name.
png(file = "boxplot.png")

# Plot the chart.
boxplot(mpg ~ cyl, data = mtcars, xlab = "Number of Cylinders",
        ylab = "Miles Per Gallon", main = "Mileage Data")

# Save the file.
dev.off()
```

Khi thực thi lệnh code trên, ta có kết quả là biểu đồ dưới đây:



9.7.2. Tạo biểu đồ boxplot với notch (rãnh)

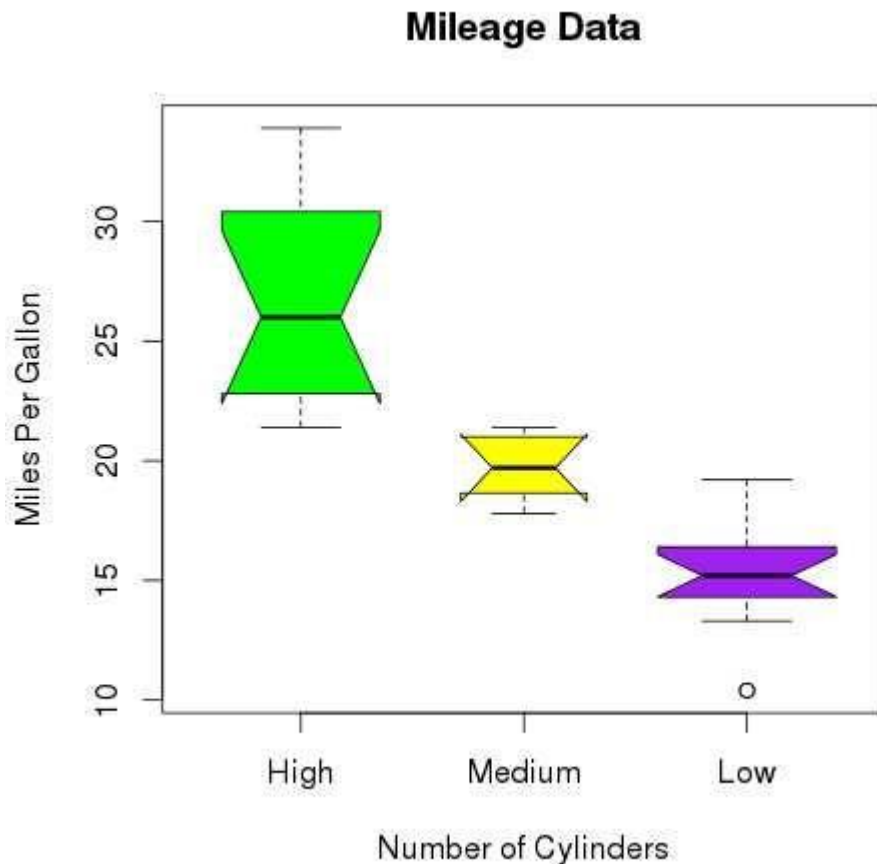
Để tìm ra giá trị trung bình giữa các nhóm dữ liệu với nhau, chúng ta sử dụng thông số notch trong boxplot.

```
# Give the chart file a name.
png(file = "boxplot_with_notch.png")

# Plot the chart.
boxplot(mpg ~ cyl, data = mtcars,
        xlab = "Number of Cylinders",
        ylab = "Miles Per Gallon",
        main = "Mileage Data",
        notch = TRUE,
        varwidth = TRUE,
        col = c("green", "yellow", "purple"),
        names = c("High", "Medium", "Low")
)
```

```
# Save the file.  
dev.off()
```

Khi thực thi đoạn code trên, ta có kết quả là biểu đồ dưới đây:



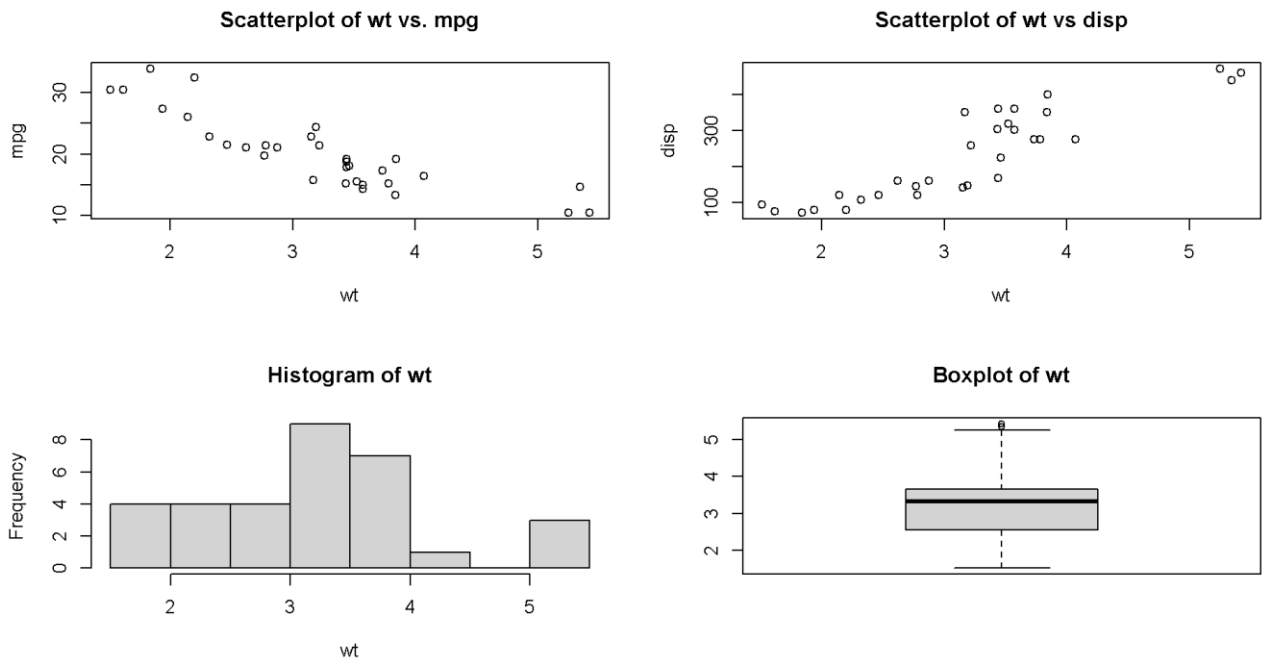
9.8. Cách vẽ nhiều biểu đồ trong R

9.8.1. Sử dụng par()

Đối với hàm `par()`, ta sử dụng tham số `mfcol` hoặc `mfrow` để đặt số hàng và số cột. Tham số `mfcol` sẽ phân bổ các đồ thị theo cột, tham số `mfrow` sẽ phân bổ các đồ thị theo hàng.

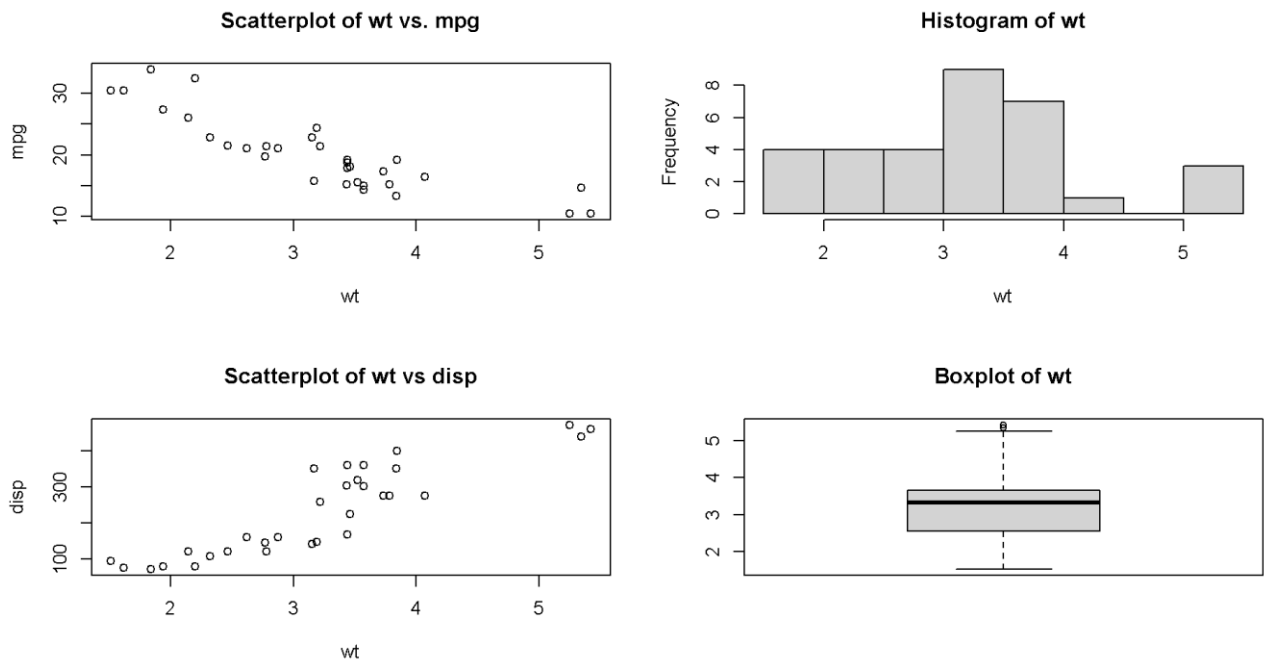
Ví dụ 1:

```
# 4 figures arranged in 2 rows and 2 columns  
attach(mtcars)  
par(mfrow=c(2,2))  
plot(wt,mpg, main="Scatterplot of wt vs. mpg")  
plot(wt,disp, main="Scatterplot of wt vs disp")  
hist(wt, main="Histogram of wt")  
boxplot(wt, main="Boxplot of wt")
```



Ví dụ 2:

```
# 4 figures arranged in 2 rows and 2 columns
attach(mtcars)
par(mfcol=c(2,2))
plot(wt,mpg, main="Scatterplot of wt vs. mpg")
plot(wt,disp, main="Scatterplot of wt vs disp")
hist(wt, main="Histogram of wt")
boxplot(wt, main="Boxplot of wt")
```



9.8.2. Sử dụng layout()

Ngoài hàm `par()` , chúng ta có thể sử dụng hàm `layout(mat)` , với `mat` là một ma trận dùng để phân chia vùng vẽ biểu đồ.

Ví dụ 3:

```
# One figure in row 1 and two figures in row 2
attach(mtcars)
mat <- matrix(c(1,1,2,3), nrow = 2, byrow = TRUE)
print(mat)
layout(mat)
hist(wt)
hist(mpg)
hist(dis)
```

